Multi-Agent DevOps Incident Analysis Suite

Design, Build & Deployment Report

AI Engineering Accelerator - Cohort 7

Wayne Johnston

June 12, 2026

Built with Claude Opus 4.6 | LangGraph | LangChain | Streamlit

Table of Contents

1. Executive Summary

The Multi-Agent DevOps Incident Analysis Suite is an AI-driven application designed for DevOps, SRE, and incident management teams. Users upload infrastructure log files (routers, switches, firewalls, servers) and a pipeline of LangGraph-orchestrated agents automatically analyzes the logs, classifies incidents, recommends remediations, generates actionable runbooks, and pushes notifications to Slack and JIRA.

The key differentiator is that the log parser is entirely AI-driven: no hardcoded regex or format-specific parsers. The LLM infers field meanings from log samples, supporting any log format including Cisco IOS syslog, CEF, RFC5424, and structured tabular formats.

Key Capabilities:

- AI-driven log format inference - no hardcoded parsers
- Multi-agent pipeline with 5 specialized agents orchestrated by LangGraph
- Automatic severity classification (critical / warning / info)
- Vendor-aware remediation with specific CLI commands
- Multi-channel Slack routing: #ops-incidents, #secops-incidents, #network-incidents
- JIRA ticket creation for critical-severity issues
- Markdown runbook generation with checklists and verification steps
- Directory watcher for automatic log ingestion
- Mock/live integration toggles for safe demo and production use

2. Design & Architecture

The system follows a pipeline architecture where six agents collaborate in sequence with conditional branching. The Orchestrator Agent, implemented as a LangGraph StateGraph, manages the flow between agents based on the severity of detected incidents.

Architecture Flow:

```
START --> Log Reader/Classifier --> Remediation --> route_by_severity has_critical
--> Slack Notifier --> JIRA Ticket Agent --> Cookbook --> END no_critical -->
Cookbook --> END
```

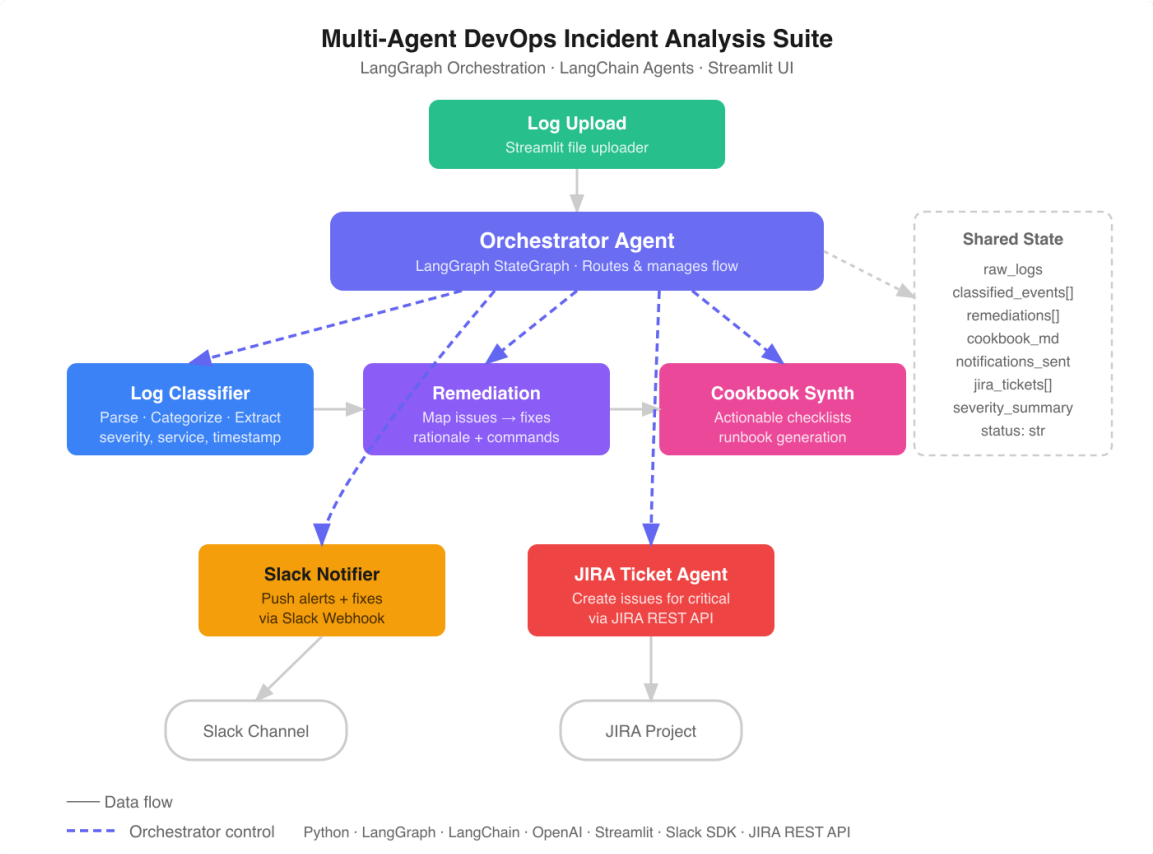

Figure 1: Multi-Agent Architecture Diagram

Each agent is implemented as a node function that reads from and writes to a shared TypedDict state (IncidentState). This allows agents to collaborate without direct coupling. The LangGraph StateGraph provides explicit control flow, conditional edges, and state management - an evolution from the simple agent patterns used in earlier cohort projects.

Design Decisions:

- **AI-driven parsing over regex**: The LLM infers field meanings from log samples, making the system format-agnostic. No new code is needed to support new log types.

• **Mock-first integrations**: Slack and JIRA start in mock mode, showing what would be sent without requiring live credentials. A sidebar toggle switches to live mode.

• **Multi-channel Slack routing**: Incidents auto-route to the appropriate channel based on category, reducing alert noise for each team.

• **Framework-agnostic core**: The agent pipeline (agents.py, graph.py, tools.py, state.py) has zero Streamlit dependencies. Only app.py imports Streamlit, enabling a future Flask or FastAPI frontend swap with no core changes.

• **Directory watcher**: In addition to manual upload, the app can monitor a directory for new .log files using watchdog, enabling automated ingestion.

3. Technology Stack

Component	Technology	Purpose
LLM	OpenAI GPT-4o via OpenRouter	Log parsing, classification, remediation
Orchestration	LangGraph StateGraph	Agent pipeline flow control
Agent Framework	LangChain Core	Tool definitions, prompt templates
UI	Streamlit	File upload, dashboard, results display
Notifications	Slack Webhook API	Multi-channel incident alerts
Ticketing	JIRA REST API	Critical issue ticket creation
File Watcher	watchdog	Directory monitoring for auto-ingestion
Environment	python-dotenv	API key management
HTTP	requests	Slack/JIRA API calls

LLM Configuration:

```
ChatOpenAI( model="openai/gpt-4o", openai_api_key=os.getenv("OPENROUTER_API_KEY"), openai_api_base="https://openrouter.ai/api/v1", temperature=0, )
```

4. Agent Design

4.1 Log Reader / Classifier Agent

The core differentiator of the system. Uses a two-stage LLM approach with no hardcoded parsers:

- **Stage 1 - Schema Inference**: Sends the first 15 lines of each log file to the LLM with the prompt: 'Analyze these log lines. Identify every field and its meaning.' Returns JSON with field positions, names, data types, and examples.
- **Stage 2 - Full Parse**: Sends the complete log text plus inferred schema to the LLM to parse every line into structured JSON events. Multi-line entries are combined.
- **Stage 3 - Classification**: Each event is classified by severity (critical/warning/info) and category (interface, auth, resource, policy, routing, security, etc.).

For large files, logs are chunked into 100-line batches. The inferred schema is cached per file to avoid redundant LLM calls. All LLM responses use response_format={"type": "json_object"} for reliable parsing.

4.2 Remediation Agent

Filters events to warning and critical severity, groups by issue type, and produces vendor-aware remediation recommendations:

- Root cause hypothesis
- Step-by-step remediation instructions
- Exact CLI commands (vendor-specific, e.g. Cisco IOS)
- Verification steps to confirm the fix
- Risk assessment of the remediation itself
- Category tag for Slack channel routing

4.3 Cookbook Synthesizer Agent

Takes all remediations and synthesizes a single, comprehensive Markdown runbook with: executive summary, priority matrix table, numbered remediation checklists with checkbox steps and code blocks for CLI commands, escalation criteria, and post-incident review items.

4.4 Slack Notification Agent

Groups remediations by category and routes each group to the appropriate Slack channel. Formats rich Slack messages with severity badges, affected systems, and issue summaries.

4.5 JIRA Ticket Agent

Creates JIRA tickets for each critical-severity issue. Ticket descriptions include root cause analysis, numbered remediation steps, and CLI commands in JIRA's {noformat} blocks. Priority is mapped from incident severity to JIRA priority levels.

5. LangGraph Orchestration

The orchestrator is implemented as a LangGraph StateGraph in graph.py. Each agent is registered as a node, and edges define the execution flow. A conditional edge after the Remediation node checks if any critical-severity issues were found:

```
graph = StateGraph(IncidentState) graph.add_node("log_reader", log_reader_node)
graph.add_node("remediation", remediation_node) graph.add_node("notification",
notification_node) graph.add_node("jira", jira_node) graph.add_node("cookbook",
cookbook_node) graph.set_entry_point("log_reader") graph.add_edge("log_reader",
"remediation") graph.add_conditional_edges("remediation", route_by_severity, {
"critical": "notification", "normal": "cookbook" }) graph.add_edge("notification",
"jira") graph.add_edge("jira", "cookbook") graph.add_edge("cookbook", END)
```

The route_by_severity function checks if any remediation has severity == 'critical'. If so, the pipeline routes through the Slack and JIRA agents before generating the cookbook. Otherwise, it skips directly to cookbook generation.

6. Slack Channel Routing

Incidents are automatically routed to the appropriate Slack channel based on their category. This reduces alert noise by ensuring each team sees only relevant incidents:

Slack Channel	Categories Routed
#ops-incidents	resource, application, database
#secops-incidents	security, auth, policy
#network-incidents	interface, routing, network, hardware

The routing logic is defined in tools.py via a CHANNEL_ROUTING dictionary. The notification_node groups remediations by target channel and sends one formatted message per channel, each containing only the issues relevant to that team.

7. Project Structure & Implementation

```
devops-incident-suite/ src/ # Application source code app.py # Streamlit UI entry point state.py # IncidentState TypedDict agents.py # All agent node functions graph.py # LangGraph StateGraph wiring tools.py # @tool wrappers (Slack, JIRA) watcher.py # Directory watcher (watchdog) log_samples/ # Synthetic test logs routers.log # Cisco IOS syslog format switches.log # Tabular switch logs security.log # CEF firewall logs servers.log # RFC5424 syslog format tests/ # Unit tests (pytest) test_state.py # State schema tests test_tools.py # Channel routing, Slack, JIRA test_watcher.py # Directory watcher tests test_agents.py # Agent node + LLM mock tests test_graph.py # Graph + pipeline tests requirements.txt .streamlit/config.toml
```

The project uses a src/tests layout. The core pipeline (state.py, agents.py, graph.py, tools.py) has zero UI dependencies, enabling a future swap from Streamlit to Flask or FastAPI without changing the agent logic.

8. Shared State Schema

All agents communicate through a shared IncidentState TypedDict. Each node returns a partial dict that is merged into the running state by LangGraph:

Field	Type	Description
raw_logs	dict[str, str]	Filename to full log text
inferred_schemas	dict[str, list]	Filename to field definitions
parsed_events	list[dict]	Unified parsed events
classified_events	list[dict]	Events with severity + category
remediations	list[dict]	Issue to fix mappings
cookbook	str	Markdown runbook content
notifications_sent	list[dict]	Slack message results
jira_tickets	list[dict]	JIRA ticket references
status	str	Current pipeline stage
errors	list[str]	Accumulated error messages

9. Synthetic Log Samples

Four synthetic log files are included for development and demo purposes. Each uses a distinct format and embeds 3-4 incidents at varying severity levels. Some incidents correlate across files (e.g., a switch port flap causing a router OSPF adjacency loss).

File	Format	Embedded Incidents
routers.log	Cisco IOS syslog	Link flaps, OSPF neighbor changes, high CPU
switches.log	Structured tabular	Port errors, STP topology changes, MAC flaps
security.log	CEF (Common Event Format)	ACL denies, auth failures, IDS alerts
servers.log	RFC5424 syslog	Disk full, OOM kills, service crashes

10. Streamlit UI

Sidebar Configuration:

- OpenRouter API key input (never pre-filled, user must enter each session)
- Directory watcher path and scan controls
- Slack Mock/Live toggle with channel routing reference
- JIRA Mock/Live toggle with URL, email, token, and project key inputs
- All credential fields start empty with placeholder text

Main Area:

- Multi-file upload accepting .log and .txt files
- 'Load Samples' button to use synthetic logs
- 'Use Watched' button to ingest from watched directory
- Log content preview with expandable sections
- 'Analyze Incidents' button (blocked until API key is provided)
- Live progress status during analysis

Results Tabs:

- **Classified Events**: Dataframe with severity color coding (red/yellow/blue)
- **Remediations**: Expandable cards per issue with CLI commands and channel routing
- **Runbook**: Rendered Markdown with download button
- **Slack Notifications**: Mock/real output grouped by channel
- **JIRA Tickets**: Ticket summaries with mock/created badges

- **Inferred Schemas**: AI-detected field definitions per log file

11. Security Considerations

- **No pre-filled credentials**: All API key and token fields start empty. Users must enter their own keys per session. Keys are never stored or persisted.
- **Secrets separation**: Server-side secrets (from .env or st.secrets) are reserved for API endpoint use only and are never exposed in the UI.
- **Analysis gating**: The Analyze button is disabled with a warning message until a valid API key is provided.
- **Mock-first integrations**: Slack and JIRA default to mock mode, preventing accidental production notifications during development and demos.
- **No sensitive data in git**: .gitignore excludes .env files, __pycache__, and IDE artifacts.

12. Deployment

GitHub Repository:

The project is hosted at github.com/wjohnston99/devops-incident-suite on the main branch.

Streamlit Cloud Deployment:

- **Step 1**: Sign in to share.streamlit.io with GitHub account
- **Step 2**: Click 'New app' and select the devops-incident-suite repository
- **Step 3**: Set branch to 'main' and main file to 'app.py'
- **Step 4**: No secrets required in Advanced Settings (users enter their own keys)
- **Step 5**: Click Deploy - app will be live within 2 minutes

Future Deployment Options:

The framework-agnostic core enables deployment on Flask, FastAPI, or any Python web framework. The user expressed interest in Flask for production deployment. Since agents.py, graph.py, tools.py, and state.py have zero Streamlit imports, a Flask api.py can import and invoke the same pipeline via HTTP endpoints.

13. Verification & Testing

Verification Steps Performed:

- All Python modules import cleanly with no errors
- LangGraph StateGraph compiles and builds without errors
- LLM connection verified via OpenRouter (GPT-4o responds correctly)
- Full pipeline tested end-to-end: 10 events parsed, 1 remediation generated, 1 Slack notification sent (mock), 1 JIRA ticket created (mock)
- Cookbook/runbook generated with proper Markdown formatting
- Channel routing verified: security maps to #secops-incidents, interface maps to #network-incidents, application maps to #ops-incidents
- Streamlit UI loads, file upload works, results tabs populate after analysis

Test Commands:

```
# Verify imports python -c "from state import IncidentState" python -c "from graph import build_incident_graph; g = build_incident_graph()" # Run the app streamlit run app.py
```

14. Risks & Mitigations

Risk	Mitigation
LLM returns malformed JSON	Use response_format={"type":"json_object"}, retry once on parse failure
Large log files exceed token budget	Chunk into 100-line batches, cache inferred schema per file
Mock mode forgotten during demo	Sidebar shows prominent MOCK/LIVE badge with color indicators
API key exposed in UI	Fields never pre-filled, secrets reserved for API endpoint only
Streamlit session state lost	Logs and results stored in st.session_state, survive button reruns

15. Future Enhancements

- **Flask API endpoint**: Add api.py with POST /api/analyze accepting log files and returning JSON results. Uses server-side secrets from .env/st.secrets.
- **RAG-based remediation**: Embed historical runbooks in FAISS so the remediation agent can retrieve past fixes for similar incidents.
- **Real-time streaming**: Stream agent progress to the UI using LangGraph's streaming interface instead of batch invoke.
- **Correlation engine**: Cross-correlate events across multiple log files to identify cascading failures (e.g., switch flap causing router OSPF loss).
- **Persistent storage**: SQLite or PostgreSQL for incident history, enabling trend analysis and recurring issue detection.
- **PagerDuty integration**: Add critical incident escalation to PagerDuty alongside Slack and JIRA.
- **Custom log format registry**: Allow users to save and share inferred schemas to speed up future parsing of known log formats.

This report was generated as part of the AI Engineering Accelerator (Cohort 7). The project demonstrates multi-agent orchestration, AI-driven log analysis, automated remediation mapping, and cross-tool notification in a scalable workflow.